Day 8 Student Notes: Claude for Git, Everyday Workflow

> **Today in one line:** Git saves your team's history. Claude writes the words and spots risks. **You check and decide.**

Git is a save-game

Word	Meaning
Repository (repo)	The project folder with its history
Commit	One saved point, with a message
Branch	A separate line of work
Merge	Join two lines of work
Pull request (PR)	A request to review a branch and merge it into main
Diff	The exact lines that changed

Six commands

Command	What it does	Changes files?
git status	What changed?	No
git diff	Show the exact changes	No
git log --oneline	List past commits	No
git add .	Pick the changes to save	No
git commit -m "..."	Save them with a message	Adds history
git merge branch	Join another branch in	Yes

status, diff and log only look. Run them freely.

Four jobs Claude does well

1. **Write:** commit messages and PR descriptions from a diff.
2. **Review:** spot risky changes in infrastructure code.
3. **Explain:** what a merge conflict means.
4. **Advise:** which branching style fits a team.

> **Rules:** Claude sees only what you paste. Remove secrets and real company data first. Claude drafts, you check and decide.

A good commit message

```
Weak:  update stuff

Good:
fix(k8s): pin shop-api image to 1.4.2

:latest changes without warning, so a
restart could deploy untested code.
```

Shape: **type(area): short summary**, a blank line, then why. Common types: fix, feat, docs, chore. Keep the first line under 60 characters. This is a common team convention, not a Git rule.

PR description and review checklist

- **Summary:** what and why
- **Changes:** the main edits
- **Risk:** what could break
- **How to test:** proof it works

Reviewer checklist: Is the change small and focused? Any secret in the diff? Is anything opened to the world? Can we roll it back?

Branching: two common styles

	Trunk-based	GitFlow
Idea	Small, short branches merged into main often	Long-lived develop, release and hotfix branches
Good for	Teams with CI and frequent deploys	Planned, versioned releases
Risk	Needs good tests and reviews	Big merges, more conflicts

Give Claude your team's facts (size, repos, how often you deploy), otherwise the advice is generic.

Merge conflicts

A conflict means two people changed the same line. Git marks the file:

```
<<<<<<< HEAD
  replicas: 3
=======
  replicas: 5
>>>>>>> feature/high-traffic
```

- **HEAD** is your side. After **=======** is their side.
- Keep one, or combine. Delete the three marker lines.
- Then **git add** the file and **git commit**.
- Stuck? **git merge --abort** goes back to before the merge.

Risky changes to look for in an infrastructure diff

In the diff	Why it is risky
0.0.0.0/0 on port 22 or a database	Opens the door to the whole internet
:latest image tag	Deploys unknown code on restart
A password or key in a file	Secrets leak through Git history
set -e removed, rm -rf $var added	Script keeps going and can delete too much
Bigger instance size or replicas	Cost jump with no mention

Hands-on (prompts also in Day08_Prompts_To_Paste.txt)

Setup: unzip **git-practice.zip**, open the folder in VS Code, open the terminal (Windows: Git Bash), run **bash setup_repo.sh**, then **cd shop-infra-practice**. No Git? Use messy.diff and conflict_example.txt from the zip.

1. Look around (change nothing)

```
git status
git log --oneline --all
git diff
git diff > my.diff
```

2. Messy diff in. Message, PR and checklist out.

```
Role: You are a senior DevOps engineer who reviews infrastructure changes.
Context: Below is a git diff from a made-up shop's infrastructure repo. It mixes
several unrelated changes.
Task: 1) Write a commit message. 2) Write a pull request description. 3) Write a
review checklist. 4) List the risky changes.
Constraints: Use only what is in the diff. Do not invent reasons. If the changes
are unrelated, say they should be split. Never repeat a secret in full.
Format: Commit: first line under 60 characters, then 2 to 3 lines on why. PR:
Summary, Changes, Risk, How to test. Checklist: 6 to 8 checkboxes. Risks: table of
file, change, why.

[paste the output of git diff here]
```

Check: did Claude flag the SSH rule, :latest, the password and the removed set -e? Did it say to split the commit? Then commit: **git add .** and **git commit -m "first line of Claude's message"**.

3. A conflict

```
git merge feature/high-traffic
git status
```

Open k8s/deployment.yaml and paste it into:

```
Role: You are a patient senior engineer.
Context: I have a merge conflict in k8s/deployment.yaml. Branch main set replicas
to 3. Branch feature/high-traffic set replicas to 5 for a sale. The file is below.
Task: Explain the conflict in simple words. Give 2 options to resolve it, say which
you recommend, and what I should ask my teammate first.
Constraints: Do not pick a winner without saying why. Under 150 words plus the
final file.
Format: Explanation, 2 options, recommendation, question to ask, final file.

[paste the conflicted file here]
```

Choose Accept Current, Accept Incoming or Accept Both in VS Code. Then **git add k8s/deployment.yaml** and **git commit**. If the merge refuses because of uncommitted changes, run **git add .** and **git commit -m "WIP"** first.

Key words

Word	Meaning
Trunk	The main branch
Conflict	Two changes to the same line that Git cannot join by itself
HEAD	Where you are now in the history
Review	A teammate checks a change before it is merged
Rollback	Going back to an earlier safe version

Check yourself

1. Name three Git commands that only look.
2. What are the two parts of a good commit message?
3. What does a merge conflict mean?
4. Name two risky things to look for in an infrastructure diff.
5. Why remove secrets before pasting a diff into Claude?

Answers

1. git status, git diff, git log.
2. A short summary line, then why the change was made.
3. Two people changed the same line and Git cannot choose.
4. Any two of: 0.0.0.0/0 on port 22, :latest tag, a password in a file, set -e removed, rm -rf with a variable, a cost jump.
5. Claude only sees what you paste, and pasted secrets leave your computer.

Take-home task (20 to 30 minutes)

1. Ask Claude prompt B1 for your own team (real or imagined) and read the five bullets.
2. Run **git merge --abort** if you are mid-merge, or make a fresh practice repo, and do the conflict again, keeping the other side.
3. Bring one risk from Claude's list that you agree with, and one you do not, with your reason.

No real company diffs, passwords or keys in any prompt.

> **Next class, Day 9:** Claude Code + Git: automated workflows (branch, edit, commit message and PR description in one go).